

leader_follower_convergence_tutorial

Leader-Follower Role Convergence: A Tutorial¶

The Setup: Imagine two people working on a project together. Each person could naturally be either a leader or a follower, but they don't know which role the other person prefers. Through repeated interactions, they need to figure out who should lead and who should follow.

The Twist: Each person forms beliefs about the other person's role, but *also* thinks about what the other person believes about their own role. This creates recursive reasoning: "I think you think I'm a leader, so I should act like one."

The Goal: Show how stable leader-follower relationships emerge through this recursive belief updating process.

This tutorial demonstrates how to use **memo** to model Theory of Mind reasoning in institutional role formation.

In [1]:

```
from memo import memo, domain
import jax
import jax.numpy as np
import matplotlib.pyplot as plt
```

Step 1: Basic Setup¶

We'll model a simple world with:

- **2 agents** (Alice and Bob)
- **2 roles** (Leader=0, Follower=1)
- **2 actions** (Assert=0, Defer=1)

The key insight: **Actions depend on both your own role AND what you think the other person expects from you.**

In [2]:

```

# Domain definitions
Roles = domain(size=2) # 0=Leader, 1=Follower
Actions = domain(size=2) # 0=Assert, 1=Defer
Agents = domain(size=2) # 0=Alice, 1=Bob

# Prior beliefs about roles (initially uncertain)
ROLE_PRIOR = np.array([0.5, 0.5]) # Equal probability of Leader/Follower

# How much each role prefers each action (when acting alone)
# Format: [role][action] = probability
INTRINSIC_PREFS = np.array([
    [0.8, 0.2], # Leaders prefer to Assert (0.8) vs Defer (0.2)
    [0.2, 0.8] # Followers prefer to Defer (0.8) vs Assert (0.2)
])

# How much people adapt their behavior based on others' expectations
SOCIAL_PRESSURE = 2.0 # Higher = more conformity to expected role

print(" Role preferences:")
print(f"   Leaders: {100*INTRINSIC_PREFS[0,0]:.0f}% Assert, {100*INTRINSIC_PREFS[0,1]:.0f}%")
print(f"   Followers: {100*INTRINSIC_PREFS[1,0]:.0f}% Assert, {100*INTRINSIC_PREFS[1,1]:.0f}%")
print(f"\n Social pressure parameter: {SOCIAL_PRESSURE}")

Role preferences:
  Leaders: 80% Assert, 20% Defer
  Followers: 20% Assert, 80% Defer

Social pressure parameter: 2.0

```

Step 2: Simple Role-Based Behavior (No Theory of Mind)¶

Let's start with the simplest case: agents act according to their true roles without considering what others think.

This gives us a baseline to compare against the recursive reasoning model.

In [3]:

```

@memo(cache=True)
def simple_behavior(role: Roles, action: Actions)(agent_id):
    """Baseline: agents act according to their intrinsic role preferences"""

    # Agent knows their own role
    world: knows(role)

    # Choose action based purely on role preferences
    action_probs = INTRINSIC_PREFS[role]
    world: chooses(action in Actions, wpp=action_probs[action])

```

```

        return Pr[world.action == action]

# Test the simple model
print(" Simple Behavior Probabilities:")
simple_probs = simple_behavior(agent_id=0) # Alice
print(f"Shape: {simple_probs.shape} (roles × actions)")
print(f"Leader acting: {simple_probs[0].round(2)} [Assert, Defer]")
print(f"Follower acting: {simple_probs[1].round(2)} [Assert, Defer]")

memo.core.MemoError: Unknown statement syntax
  file: "469816499.py", line 9, in @memo simple_behavior
    action_probs = INTRINSIC_PREFS[role]
    ^

hint: The full statement is Assign(targets=[Name(id='action_probs',
      ctx=Store())],
      value=Subscript(value=Name(id='INTRINSIC_PREFS', ctx=Load()),
        slice=Name(id='role', ctx=Load()), ctx=Load()))

info: You are using memo 1.2.3, JAX 0.7.1, Python 3.12.11 on Darwin.

```

Step 3: Theory of Mind - "What do you think I am?" ¶

Now for the interesting part! We'll add recursive reasoning where:

1. **Alice** has beliefs about **Bob's** role
2. **Alice** also thinks about what **Bob** believes about **Alice's** role
3. **Alice's** **actions** depend on both her intrinsic preferences AND what she thinks Bob expects

This is where **memo** really shines - it handles the "I think you think I think..." recursion naturally.

In []:

```

@memo(cache=True)
def recursive_behavior(my_role: Roles, other_role: Roles, action: Actions)(agent_id, other_id):
    """Agent acts based on intrinsic preferences AND what they think other expects"""

    # Base case: if no theory of mind, just use intrinsic preferences
    if level == 0:
        world: knows(my_role)
        action_probs = INTRINSIC_PREFS[my_role]
        world: chooses(action in Actions, wpp=action_probs[action])
        return Pr[world.action == action]

    # Recursive case: consider what other thinks about me

```

```

me: knows(my_role) # I know my true role

# What do I think about the other person's role?
me: chooses(other_role in Roles, wpp=ROLE_PRIOR[other_role])

# What does the other person think about MY role?
me: thinks[
    other: knows(other_role), # Other knows their own role
    other: chooses(belief_about_me in Roles, wpp=ROLE_PRIOR[belief_about_me]) # Other's
]

# Now I act considering:
# 1. My intrinsic preferences for my true role
# 2. What I think the other expects from me (based on what they think my role is)
intrinsic_prefs = INTRINSIC_PREFS[my_role]
expected_prefs = INTRINSIC_PREFS[me.other.belief_about_me] # What other expects based on
# Weighted combination of intrinsic and social expectations
combined_prefs = intrinsic_prefs + SOCIAL_PRESSURE * expected_prefs
combined_probs = combined_prefs / combined_prefs.sum() # Normalize to probabilities

me: chooses(action in Actions, wpp=combined_probs[action])

return Pr[me.my_role == my_role and me.other_role == other_role and me.action == action]

print(" Recursive reasoning model defined!")
print("   Now agents consider: 'What does the other person expect from me?'")

```

Step 4: Comparing Simple vs Recursive Behavior ¶

Let's see how adding Theory of Mind changes behavior. We expect:

- **Simple model:** Agents stick to their intrinsic role preferences
- **Recursive model:** Agents modify their behavior based on social expectations

In []:

```

# Compare simple vs recursive behavior
print(" Behavior Comparison")
print("=" * 50)

# Simple behavior (no theory of mind)
simple_probs = simple_behavior(agent_id=0)
print("\n Simple Model (no theory of mind):")
print(f"   If Leader: {100*simple_probs[0,0]:.0f}% Assert, {100*simple_probs[0,1]:.0f}% Defect")
print(f"   If Follower: {100*simple_probs[1,0]:.0f}% Assert, {100*simple_probs[1,1]:.0f}% Defect")

```

```

# Recursive behavior (with theory of mind)
recursive_probs = recursive_behavior(agent_id=0, other_id=1, level=1)
print(f"\n Recursive Model (level 1 theory of mind):")
print(f"    Shape: {recursive_probs.shape} (my_role × other_role × action)")

# Marginalize over other's role to get action probabilities for each of my roles
recursive_marginal = recursive_probs.sum(axis=1) # Sum over other_role dimension
recursive_marginal = recursive_marginal / recursive_marginal.sum(axis=1, keepdims=True) # M

print(f"    If Leader: {100*recursive_marginal[0,0]:.0f}% Assert, {100*recursive_marginal[0,1]:.0f}% Defer")
print(f"    If Follower: {100*recursive_marginal[1,0]:.0f}% Assert, {100*recursive_marginal[1,1]:.0f}% Defer")

# Show the effect of social pressure
print(f"\n Theory of Mind Effect:")
leader_change = recursive_marginal[0,0] - simple_probs[0,0]
follower_change = recursive_marginal[1,1] - simple_probs[1,1]
print(f"    Leaders now Assert {leader_change:+.1%} more")
print(f"    Followers now Defer {follower_change:+.1%} more")

```

Step 5: Multi-Round Interaction and Convergence ¶

Now let's simulate multiple rounds of interaction where:

1. Agents act based on recursive reasoning
2. They observe each other's actions
3. They update their beliefs about each other's roles
4. Over time, beliefs should converge to stable role assignments

This shows how **institutions emerge** through repeated social interaction!

In []:

```

@memo(cache=True)
def belief_update(new_belief: Roles)(prior_belief, observed_action, assumed_role):
    """Update belief about someone's role based on their observed action"""

    # Prior belief about their role
    world: chooses(true_role in Roles, wpp=prior_belief[true_role])

    # Likelihood: how probable is the observed action given each possible role?
    action_likelihood = INTRINSIC_PREFS[assumed_role, observed_action]

    # Posterior belief (unnormalized)
    posterior_weight = prior_belief[assumed_role] * action_likelihood

    return posterior_weight

```

```

def simulate_interaction_round(alice_role, bob_role, alice_belief_about_bob, bob_belief_about_alice, level):
    """Simulate one round of interaction between Alice and Bob"""

    # Alice acts based on recursive reasoning
    alice_action_probs = recursive_behavior(agent_id=0, other_id=1, level=level)
    # Marginalize and get action probabilities for Alice's actual role
    alice_marginal = alice_action_probs[alice_role].sum(axis=0) # Sum over Bob's role
    alice_marginal = alice_marginal / alice_marginal.sum()
    alice_action = np.random.choice(2, p=alice_marginal)

    # Bob acts based on recursive reasoning
    bob_action_probs = recursive_behavior(agent_id=1, other_id=0, level=level)
    bob_marginal = bob_action_probs[bob_role].sum(axis=0) # Sum over Alice's role
    bob_marginal = bob_marginal / bob_marginal.sum()
    bob_action = np.random.choice(2, p=bob_marginal)

    # Update beliefs based on observed actions
    # Alice updates belief about Bob
    alice_new_belief = np.array([
        belief_update(alice_belief_about_bob, bob_action, 0)[0], # P(Bob is Leader | action)
        belief_update(alice_belief_about_bob, bob_action, 1)[0] # P(Bob is Follower | action)
    ])
    alice_new_belief = alice_new_belief / alice_new_belief.sum() # Normalize

    # Bob updates belief about Alice
    bob_new_belief = np.array([
        belief_update(bob_belief_about_alice, alice_action, 0)[0], # P(Alice is Leader | action)
        belief_update(bob_belief_about_alice, alice_action, 1)[0] # P(Alice is Follower | action)
    ])
    bob_new_belief = bob_new_belief / bob_new_belief.sum() # Normalize

    return alice_action, bob_action, alice_new_belief, bob_new_belief

print(" Multi-round interaction simulator ready!")
print(" Agents will act → observe → update beliefs → repeat")

In [ ]:

# Simulate convergence process
def simulate_convergence(alice_true_role, bob_true_role, n_rounds=10, seed=42):
    """Simulate multiple rounds and track belief convergence"""

    np.random.seed(seed)

    # Initial beliefs (uncertain)
    alice_belief_about_bob = ROLE_PRIOR.copy()
    bob_belief_about_alice = ROLE_PRIOR.copy()

```

```

# Track history
history = {
    'round': [],
    'alice_action': [],
    'bob_action': [],
    'alice_belief_bob_is_leader': [],
    'bob_belief_alice_is_leader': []
}

role_names = ['Leader', 'Follower']
action_names = ['Assert', 'Defer']

print(f" Starting simulation: Alice is {role_names[alice_true_role]}, Bob is {role_names[bob_true_role]}")
print("\n" + "="*80)

for round_num in range(n_rounds):
    # Simulate interaction
    alice_action, bob_action, alice_new_belief, bob_new_belief = simulate_interaction_round(
        alice_true_role, bob_true_role, alice_belief_about_bob, bob_belief_about_alice
    )

    # Record history
    history['round'].append(round_num)
    history['alice_action'].append(alice_action)
    history['bob_action'].append(bob_action)
    history['alice_belief_bob_is_leader'].append(alice_new_belief[0])
    history['bob_belief_alice_is_leader'].append(bob_new_belief[0])

    # Print round summary
    print(f"Round {round_num+1}:")
    print(f"   Actions: Alice {action_names[alice_action]:>6}, Bob {action_names[bob_action]:>6}")
    print(f"   Alice thinks Bob is Leader: {alice_new_belief[0]:.2%}")
    print(f"   Bob thinks Alice is Leader: {bob_new_belief[0]:.2%}")
    print()

    # Update beliefs for next round
    alice_belief_about_bob = alice_new_belief
    bob_belief_about_alice = bob_new_belief

return history

# Run simulation
history = simulate_convergence(alice_true_role=0, bob_true_role=1, n_rounds=8)

In [ ]:

```

```

# Visualize convergence
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

rounds = np.array(history['round']) + 1

# Plot 1: Belief evolution
ax1.plot(rounds, history['alice_belief_bob_is_leader'], 'b-o', label='Alice thinks Bob is Leader')
ax1.plot(rounds, history['bob_belief_alice_is_leader'], 'r-s', label='Bob thinks Alice is Leader')
ax1.axhline(y=0.5, color='gray', linestyle='--', alpha=0.5, label='Uncertain (50%)')
ax1.set_xlabel('Round')
ax1.set_ylabel('Belief Probability')
ax1.set_title('Belief Convergence Over Time')
ax1.legend()
ax1.grid(True, alpha=0.3)
ax1.set_ylim(0, 1)

# Plot 2: Action patterns
alice_actions = np.array(history['alice_action'])
bob_actions = np.array(history['bob_action'])

ax2.scatter(rounds, alice_actions + 0.05, c='blue', s=100, alpha=0.7, label='Alice', marker='o')
ax2.scatter(rounds, bob_actions - 0.05, c='red', s=100, alpha=0.7, label='Bob', marker='s')
ax2.set_xlabel('Round')
ax2.set_ylabel('Action')
ax2.set_title('Action Patterns Over Time')
ax2.set_yticks([0, 1])
ax2.set_yticklabels(['Assert', 'Defer'])
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Summary statistics
final_alice_belief = history['alice_belief_bob_is_leader'][-1]
final_bob_belief = history['bob_belief_alice_is_leader'][-1]

print("\n Final Convergence Results:")
print(f"    Alice believes Bob is Leader: {final_alice_belief:.1%}")
print(f"    Bob believes Alice is Leader: {final_bob_belief:.1%}")
print(f"    \n Role assignment stability: {'HIGH' if abs(final_alice_belief - 0.5) > 0.3 else 'LOW'}")

```

Step 6: Key Insights and Extensions¶

What We've Shown¶

1. **Recursive Reasoning Matters:** Agents who consider "what do you think I am?" behave differently than those who just follow intrinsic preferences
2. **Institutional Convergence:** Through repeated interaction and belief updating, initially uncertain agents develop stable role expectations
3. **Self-Fulfilling Prophecies:** When Alice thinks Bob expects her to lead, she acts more like a leader, which reinforces Bob's belief that she IS a leader

Why This Matters for Institutional Research¶

- **Role Emergence:** Shows how leadership isn't just individual traits, but emerges through social expectations
- **Institutional Stability:** Demonstrates why social roles become "sticky" once established
- **Theory of Mind:** Captures the recursive nature of social reasoning that standard models miss

Extensions You Could Add¶

- **Multiple roles:** Leader/Follower/Neutral/Specialist
- **Group dynamics:** 3+ agents with coalition formation
- **Role conflict:** What happens when both agents want to lead?
- **Institutional memory:** How do role expectations persist when new agents join?
- **Power dynamics:** Asymmetric costs/benefits of different role assignments

Why Memo is Perfect for This¶

Compare implementing this recursive reasoning in other frameworks:

PyMC/Stan: Can't handle "I think you think I think" recursion naturally

Standard RL: Requires manually coding belief states and updates

Game Theory: Assumes agents optimize utility, missing the belief formation process

Memo: The `me: thinks[other: thinks[me: ...]]` syntax directly captures the recursive structure of social reasoning!

For institutional convergence research, this recursive reasoning capability is essential because institutions ARE recursive belief structures - they exist because "everyone knows that everyone knows" the rules.

In []:

```
# Experiment: What happens with different social pressure levels?
print(" Experiment: Social Pressure Effects")
print("="*50)
```

```
pressure_levels = [0.5, 1.0, 2.0, 5.0]
convergence_speeds = []
```

```
for pressure in pressure_levels:
    # Temporarily change social pressure
    global SOCIAL_PRESSURE
    SOCIAL_PRESSURE = pressure

    # Run simulation
    hist = simulate_convergence(alice_true_role=0, bob_true_role=1, n_rounds=10, seed=42)

    # Measure convergence (how quickly beliefs become confident)
    final_confidence = abs(hist['alice_belief_bob_is_leader'][-1] - 0.5)
    convergence_speeds.append(final_confidence)
```

```
print(f"    Pressure {pressure:3.1f}: Final confidence = {final_confidence:.2f}")
```

```
# Reset to original value
SOCIAL_PRESSURE = 2.0
```

```
print(f"\n Insight: Higher social pressure → {'Faster' if convergence_speeds[-1] > converge
```